EstateFlow
Complete Real Estate Management System

A modern property sales, rental and operations platform

SUBMI TTE D BY

Name	**Nelamoni**
ID	1297020
Batch	PWAD/USSL-M/73/01
Programme	IsDB-BISEW IT Scholarship Programme
Round	73

Technology: HTML • CSS • JavaScript • PHP • MySQL

Document purpose: Personal project planning, system design and development presentation.

VISION

Make property discovery, booking, rental operations and financial tracking simpler, clearer and more reliable from one integrated web platform.

Introduction & Project Background

Introduction

EstateFlow is a proposed web-based real estate management platform designed to bring property listings, customer inquiries, property visits, bookings, rentals, sales records and administration into a single system. It combines a public-facing property discovery website with secure role-based dashboards for operational users.

Background

Property businesses often coordinate information through phone calls, spreadsheets, paper agreements and separate messaging apps. As the number of properties and customers grows, these disconnected workflows make it harder to locate current records, track availability and follow up on payments. EstateFlow will introduce a structured digital workflow supported by a relational database.

Why build it?

- Centralize accurate property and customer information.
- Reduce manual follow-ups and duplicate records.
- Make property availability and booking status easier to understand.

- Provide a practical portfolio project demonstrating end-to-end web engineering.

Project purpose

The primary purpose is to design and implement a maintainable, secure and expandable real estate web application that can be tested with sample data and later adapted to a real agency or independent property owner.

Problem Statement, Goals & Success

Problem statement

Without a shared system, an agent may advertise a property already booked, a tenant payment may be recorded late, and management may not know which inquiries require attention. Paper-based records also make historical reporting and audit trails difficult. EstateFlow addresses these operational gaps by keeping a consistent record of properties, users, bookings, contracts and transactions.

Main objective

Develop an integrated, role-based web application for property discovery and day-to-day real estate operations.

Specific objectives

- Implement secure registration, login and user permissions.
- Allow authorized users to create and maintain property listings with photos.
- Provide search, filtering, inquiries and visit requests for customers.
- Track rental agreements, rent dues, payments and maintenance requests.
- Produce useful dashboards and downloadable operational reports.
- Build clean APIs and a normalized MySQL schema for future growth.

Measurable completion criteria

Measure	Initial target
Core user roles	Admin, owner, agent, customer/tenant
Core workflow	Property → inquiry → visit → booking
Data integrity	Validated forms, foreign keys and status rules
Testing	Happy-path and error-path tests for each module
Usability	Responsive layouts for mobile and desktop

Target Users & Value Proposition

User	Needs and benefits
Property owners	Monitor listings, occupancy, rental income and maintenance.
Real estate agencies	Coordinate inventory, agents, inquiries and sales pipeline.
Agents	Track assigned listings, customer visits and commissions.
Buyers	Compare listings, request viewings and track inquiries.
Tenants	View agreements, payment history and maintenance updates.
System administrator	Manage users, permissions, data quality and reports.

Representative scenarios

- A buyer searches for a two-bedroom apartment within a budget and requests a visit.
- An agent confirms a visit and updates the inquiry status.
- A property owner reviews available units and monthly rental performance.
- A tenant submits a maintenance ticket and follows its status.

Project boundaries

EstateFlow is initially a management and discovery application, not a legally authoritative land registry or automated title-verification service. Online payment collection and legal document verification will be introduced only after appropriate integration and compliance review.

Functional Modules & User Journeys

Module	Key functions	Priority
Authentication	Register, login, logout, password reset, role access	MVP
Property inventory	Create/edit listings, images, pricing, status	MVP
Public discovery	Search, filters, details, contact	MVP
Booking & visits	Visit request, agent approval, schedule	MVP
Customers & agents	Profiles, inquiry ownership, follow-up	Phase 2
Rental operations	Agreements, monthly dues, receipts	Phase 2
Sales & finance	Sales records, installments, reports	Phase 2
Maintenance	Issue tickets, assignment, status	Phase 2
Notifications	Booking and due-date alerts	Phase 3

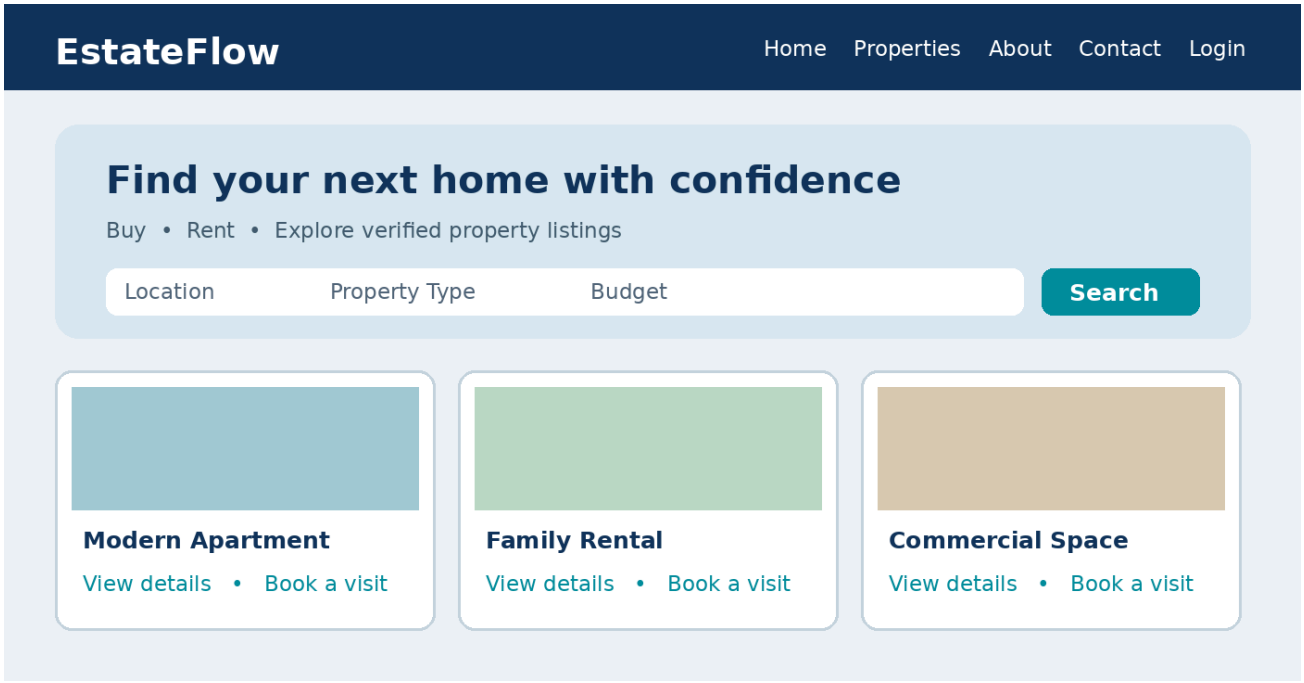
Typical customer journey

Open home page → search by location and price → view property details → register or log in → request visit → receive booking status → contact agent.

Typical admin journey

Log in → review dashboard → approve listings → assign agent → review booking queue → update property status → inspect reports.

Homepage Design & Page Structure



Homepage layout

1. Header: EstateFlow logo, Home, Properties, About, Contact and Login. 2. Hero section: clear value statement and property search. 3. Featured properties: image, price, area, location and availability. 4. How it works: Search → View → Book. 5. Benefits and trust information. 6. Footer: contact, policies and support links.

Other key screens

Property listing, property detail, registration/login, customer dashboard, agent workspace, admin dashboard, booking calendar, rent ledger, maintenance tickets and reporting.

Design guidelines

Use a navy-and-teal visual identity, consistent spacing, readable typography, clear status labels, responsive cards, keyboard-accessible forms and loading/empty/error states. Avoid placing private owner or tenant information on public pages.

Technology Stack & System Architecture

Layer	Technology	Purpose
Structure	HTML5	Semantic markup and accessible forms
Presentation	CSS3	Responsive layout, branding and components
Browser logic	JavaScript	Form handling and interactions
Frontend application	React	Reusable components, routing and state
Server language	PHP	Backend programming language
Backend framework	Laravel	REST API, validation, authentication, business rules
Database	MySQL	Relational persistent storage
Development tools	Git, GitHub, VS Code	Source control and development workflow

Request flow

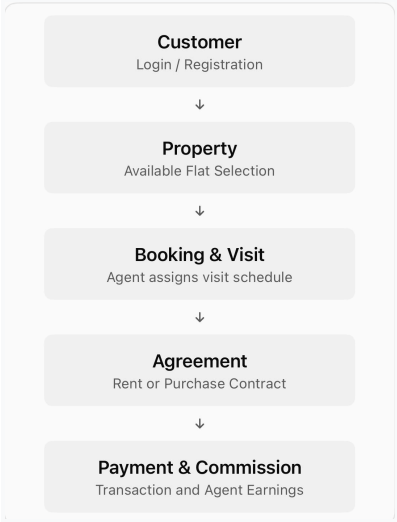
Browser / React UI → HTTPS request → MySQL

Suggested integrations

Laravel Sanctum for first-party SPA authentication, a secure file storage strategy for images, charting for analytics and optional email notifications. Use environment variables for secrets and never commit them to Git.

Relationships Work

How will the relationship work for example, if a customer wants to visit a flat

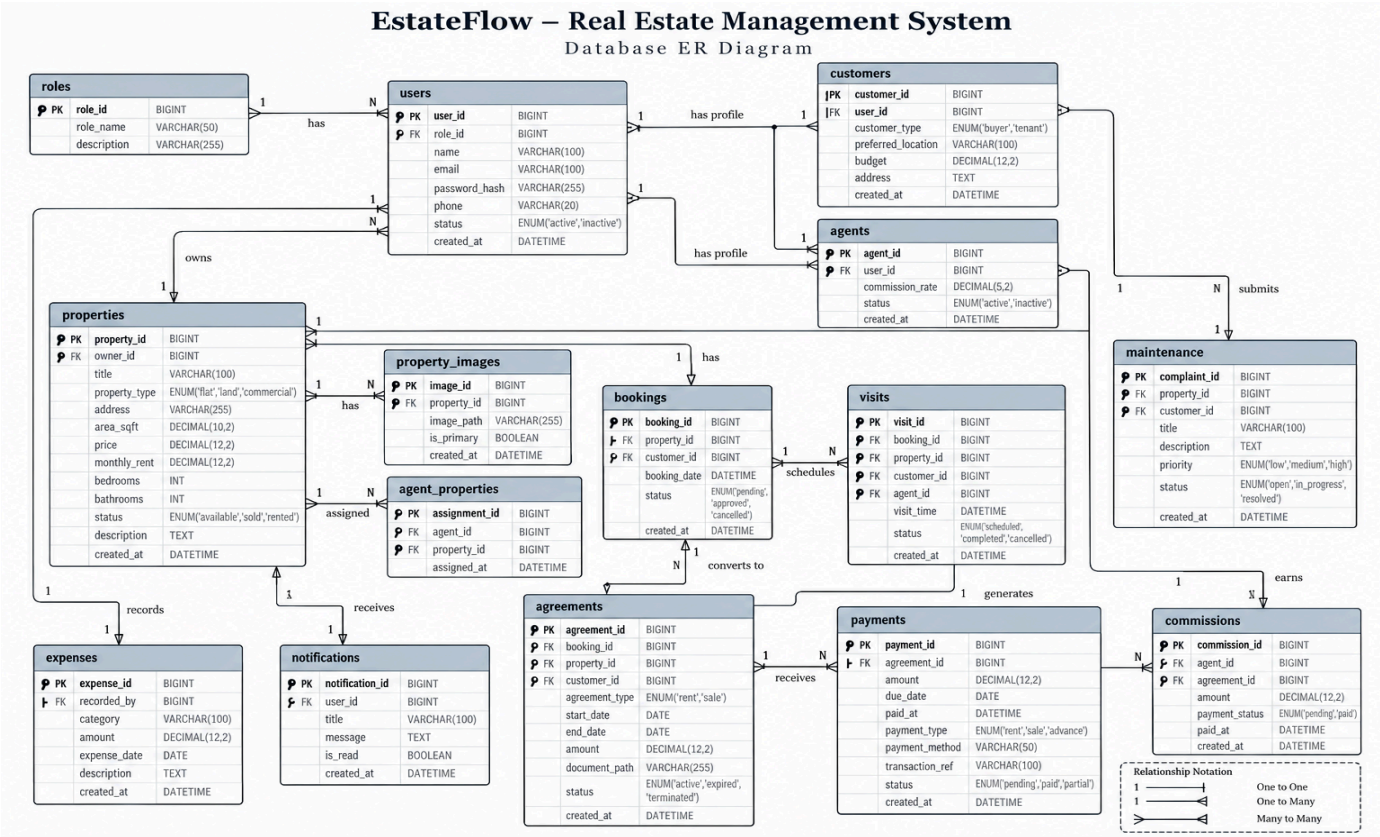


Database ER Diagram & Relationships

The diagram below is a conceptual overview. The implementation will also include indexes, timestamps, unique constraints and lookup/status rules.

Relationship examples

One owner can own many properties. One property can have



multiple images, bookings, maintenance requests and expense records. One rental agreement can generate many payment records. Users may have customer, owner or agent profiles, and authorization determines which records they can access.

Database Schema & Integrity Rules

Entity	Core fields
users	id PK, name, email UNIQUE, password_hash, role
owners / agents	id PK, user_id FK UNIQUE, contact fields
properties	id PK, owner_id FK, title, address, type, price, status
property_images	id PK, property_id FK, path, sort_order
bookings	id PK, property_id FK, customer_id FK, visit_at, status
rental_agreements	id PK, property_id FK, tenant_id FK, start/end, rent
payments	id PK, agreement_id FK, amount, due_at, paid_at, status
sales	id PK, property_id FK, buyer_id FK, price, sold_at
maintenance_requests	id PK, property_id FK, reported_by FK, issue, status
notifications / expenses	user_id FK / property_id FK, type, amount, metadata

Data integrity

- Use InnoDB foreign keys, indexes on search fields and database migrations.
- Use DECIMAL for money, not floating-point numbers.
- Use transactions and locking when confirming bookings or recording sales.

- Prevent overlapping active rental agreements for the same unit through application rules and transactional checks.
- Use soft deletes where records must remain available for financial history.

Security requirements

Apply Laravel validation and policies, secure password hashing, rate limiting, CSRF protection for cookie-based authentication, safe image upload validation, HTTPS, restricted CORS and least-privilege permissions. Store audit-relevant changes and avoid exposing personal details in API responses.

Testing approach

Unit tests for calculations and status rules; feature tests for API authentication and permissions; database tests for constraints; end-to-end tests for property search and booking; responsive testing across screen sizes. Include invalid-input, unauthorized-access and duplicate-booking scenarios.

Practical milestone

At the end of the first month, a visitor should be able to open a responsive homepage, browse demo properties, apply basic filters and view a property detail page.

Payment Management Module — Detailed Design

This section extends the EstateFlow personal project proposal with a practical, auditable payment workflow for rent, property sales, booking deposits and installment payments. Payments will be recorded and verified by the backend; a production gateway can be integrated later.

production gateway can be integrated later

- Customer can view invoices, due dates, paid amounts, outstanding balance and receipts.
- Admin can create invoices, record manual payments, confirm payment evidence and issue refunds where permitted.
- Supported payment purposes: rental invoice, sale installment, booking deposit and maintenance charge (if billed).
- Payment statuses: pending, processing, paid, failed, cancelled, refunded and partially_paid (invoice-level).
- Store payment reference, method, currency (BDT), timestamps and gateway transaction ID; never store card PIN, CVV or payment credentials.
- A payment is marked paid only after server-side gateway verification or authorized manual confirmation; client-side success screens are not proof of payment.
- Use database transactions, unique external transaction IDs and idempotency keys to prevent double-crediting.

Payment Lifecycle

Invoice created → Customer chooses payment method → Payment attempt created (pending) → Gateway/manual verification → Payment confirmed or failed → Invoice balance recalculated → Receipt generated → Customer and admin notified.

Payment-Related Database Tables

Table	Core fields	Purpose
invoices	invoice_id PK, customer_id FK, agreement_id FK NULL, sale_id FK NULL, total_amount, due_date, status	Bill and balance tracking
payments	payment_id PK, invoice_id FK, customer_id FK, amount, method, status, paid_at	Confirmed or pending payment records
payment_transactions	transaction_id PK, payment_id FK, provider, external_reference UNIQUE, idempotency_key UNIQUE, verification_status	Audit and gateway callbacks
refunds	refund_id PK, payment_id FK, amount, reason, status, approved_by FK	Refund controls

Integrity rules: DECIMAL(12,2) for monetary amounts; CHECK amount > 0; indexed foreign keys; paid totals cannot exceed the invoice amount unless an explicit credit/refund policy applies. Enforce the rules in Laravel service-layer transactions.

Deployment, Risks & Final Vision

Deployment plan

Use a Laravel-compatible PHP hosting environment, a managed or hosted MySQL database, HTTPS, a build pipeline for React assets, environment-specific configuration and scheduled backups. Separate development, staging and production environments when feasible.

Risks and mitigations

Risk	Mitigation
Scope grows too quickly	Finish the MVP before advanced modules
Duplicate bookings	Transactional status checks and tests
Sensitive information exposure	Role permissions, validation and minimal API fields
Lost data	Backups, migration history and restore tests
Slow search	Indexes, pagination and optimized queries
Poor mobile experience	Mobile-first design and device testing

Expected outcomes

A working personal full-stack portfolio application, an inspectable MySQL data model, reusable React UI, Laravel API documentation, automated tests and a demonstration workflow covering property listing through booking.

Future expansion

Interactive map search, multilingual support, owner analytics, agent commission automation, payment gateway integration, PDF agreements and mobile application support can be planned after the first stable release.

Presentation summary

EstateFlow aims to solve fragmented property management by combining searchable listings, booking workflows and operational dashboards in one platform. Its React frontend, Laravel API and MySQL database provide a clear foundation for an extensible real estate product.